

Les fondations de l'informatique : du matériel au cloud

Oct 2, 2026 · @Navatech

Comment utiliser ce guide

Tout ce que tu vas faire en DevOps/Cloud est un empilement de couches : chaque couche cache la complexité de celle du dessous. Comprendre l'empilement, c'est comprendre pourquoi chaque outil existe.

1. **Matériel** : des puces qui calculent, stockent et transmettent des bits.
2. **Système d'exploitation** : le logiciel qui partage ce matériel entre les programmes.
3. **Réseau** : comment deux machines se parlent (TCP/IP).
4. **Virtualisation** : plusieurs machines « logiques » sur une seule machine physique.
5. **Cloud** : la virtualisation louée à la demande, pilotée par API, à l'échelle mondiale.
6. **Conteneurs (Docker)** : isoler une application, pas une machine entière.
7. **Orchestration (Kubernetes)** : piloter des centaines de conteneurs sur plusieurs machines.

Lis le guide dans l'ordre une première fois (2 à 3 h), puis reviens sur la section qui correspond à ta semaine du parcours NavMarket : S1-2 → sections 2-3, S3-4 → réseau, S7 → virtualisation, S11 → Docker, S13 → cloud, S19 → Kubernetes. Chaque section renvoie aux sources officielles listées à la fin.

1. L'ordinateur et le hardware

Un ordinateur fait trois choses : il **calcule** (CPU), il **retient** (RAM et stockage) et il **communique** (réseau, périphériques). Un serveur est le même objet, optimisé pour tourner 24 h/24 sans écran.

Composant	Rôle	Ce qui compte pour toi en ops
CPU (processeur)	Exécute les instructions. Plusieurs cœurs = plusieurs tâches en parallèle.	Nombre de cœurs/threads, extensions de virtualisation Intel VT-x / AMD-V (indispensables pour KVM). Dans le cloud on parle de vCPU .

Composant	Rôle	Ce qui compte pour toi en ops
RAM (mémoire vive)	Stocke les données en cours d'usage. Très rapide, effacée à l'extinction.	Manque de RAM = la machine « swappe » ou le noyau tue des processus (OOM killer).
Stockage (HDD, SSD, NVMe)	Garde les données de façon durable.	Capacité, débit, IOPS (opérations par seconde). Les bases de données y sont très sensibles.
Carte mère et bus (PCIe)	Relie tous les composants.	Détermine ce qu'on peut brancher et à quelle vitesse.
Carte réseau (NIC)	Envoie et reçoit des trames sur un câble ou en Wi-Fi.	Porte une adresse MAC unique ; débit 1, 10, 25, 100 Gbit/s en datacenter.
GPU	Calcul massivement parallèle.	Affichage, mais surtout IA et calcul scientifique.
Firmware BIOS / UEFI	Premier programme lancé au démarrage.	Active VT-x, gère Secure Boot (tu l'as croisé avec le dual boot).

Le démarrage d'une machine, dans l'ordre : mise sous tension → UEFI teste le matériel → UEFI lance le chargeur de démarrage (GRUB sous Ubuntu) → GRUB charge le **noyau** Linux en RAM → le noyau démarre le premier processus (**systemd**, PID 1) → systemd lance les services (SSH, Nginx...).

Bits et unités : 1 octet = 8 bits. Le stockage se compte en octets (Go, To), le débit réseau en bits par seconde (Mbit/s). Une fibre à 1 Gbit/s transfère donc au mieux ~125 Mo/s.

Serveur vs PC : un serveur a de la RAM ECC (corrige les erreurs), des alimentations et disques redondants (RAID), une carte de gestion à distance (iDRAC, iLO) et se range dans une baie (rack) de datacenter. C'est sur ces machines que tourne le cloud.

2. Software et système d'exploitation

Le système d'exploitation (OS) est l'arbitre entre le matériel et les programmes : il décide qui utilise le CPU, la RAM, le disque et le réseau, et empêche les programmes de se marcher dessus.

Les familles de logiciels

- **Firmware** : gravé près du matériel (UEFI, firmware d'un disque).
- **Système** : noyau, pilotes (drivers), services de base.
- **Applications** : Nginx, PostgreSQL, ton navigateur, ton app NestJS.

- **Bibliothèques et runtimes** : code partagé (glibc, OpenSSL) ou environnement d'exécution (Node.js, Python, JVM).

Espace noyau et espace utilisateur. Le **noyau** (kernel) a tous les droits sur le matériel. Les programmes tournent en **espace utilisateur** et doivent passer par des **appels système** (syscalls : `open`, `read`, `fork`, `socket` ...) pour demander un service au noyau. Retiens bien cette frontière : c'est exactement ce qui distingue une VM d'un conteneur (section 8).

Les 5 missions de l'OS

1. **Processus** : un programme en cours d'exécution, avec un PID, un propriétaire, de la mémoire. L'**ordonnanceur** partage le CPU entre eux des milliers de fois par seconde. Commandes : `ps`, `top`, `htop`, `kill`.
2. **Mémoire** : chaque processus croit avoir sa propre mémoire (**mémoire virtuelle**) ; le noyau la traduit vers la RAM réelle et déborde sur le disque (**swap**) si besoin. Commande : `free -h`.
3. **Fichiers** : le système de fichiers (ext4, XFS, NTFS) organise le disque en dossiers et fichiers avec des **permissions** (`rw`, propriétaire, groupe). Sous Linux, « tout est fichier », même les périphériques (`/dev/sda`). Commandes : `ls -l`, `chmod`, `chown`, `df -h`.
4. **Périphériques** : les pilotes traduisent les ordres du noyau pour chaque matériel.
5. **Réseau et sécurité** : la pile TCP/IP vit dans le noyau ; les utilisateurs, groupes et `sudo` contrôlent qui peut faire quoi.

Linux vs Windows

Critère	Linux (Ubuntu, Debian, RHEL)	Windows / Windows Server
Licence	Libre et gratuit (sauf support RHEL)	Payant
Usage serveur	Très majoritaire dans le cloud et le web	Annuaire Active Directory, applis .NET historiques
Administration	Ligne de commande (Bash), fichiers texte dans <code>/etc</code>	Interface graphique + PowerShell
Conteneurs	Natifs (Docker est né sur Linux)	Via WSL2 ou conteneurs Windows

Arborescence Linux à connaître : `/etc` (configuration), `/var/log` (journaux), `/home` (utilisateurs), `/usr/bin` (programmes), `/tmp` (temporaire), `/proc` (infos du noyau en direct).

Le shell (Bash) est un programme qui lit tes commandes et les fait exécuter par le noyau. Un **service** (ou démon) est un processus qui tourne en arrière-plan, géré par systemd :
`systemctl status nginx`, `journalctl -u nginx`.

3. Les bases du réseau

Un réseau, c'est des machines qui échangent des messages selon des règles communes appelées **protocoles**. Pour ne pas tout mélanger, on découpe le problème en **couches** : chaque couche rend un service à celle du dessus et ignore les détails de celle du dessous.

Modèle OSI (7 couches, théorique) et modèle TCP/IP (4 couches, celui d'Internet)

OSI	TCP/IP	Ce qu'elle fait	Unité	Adresse	Exemples	Équipement
7 Application, 6 Présentation, 5 Session	Application	Le service rendu à l'utilisateur, chiffrement, format	Données	Nom (DNS), URL	HTTP(S), DNS, SSH, TLS, SMTP	Serveur, proxy, load balancer L7
4 Transport	Transport	Livre les données au bon programme , de façon fiable ou non	Segment (TCP) / datagramme (UDP)	Port (80, 443, 22)	TCP, UDP	Pare-feu, load balancer L4
3 Réseau	Internet	Achemine d'un réseau à l'autre, à travers le monde	Paquet	Adresse IP	IPv4, IPv6, ICMP (ping)	Routeur
2 Liaison	Accès réseau	Livre au voisin direct sur le même réseau local	Trame	Adresse MAC	Ethernet, Wi-Fi, ARP, VLAN	Switch
1 Physique	Accès réseau	Transporte les bits	Bit	—	Câble cuivre, fibre, ondes radio	Câble, répéteur

Moyen mémotechnique OSI de bas en haut : **Pour Le Réseau Tout Se Passe** Automatisé.

L'encapsulation : quand ton navigateur envoie une requête HTTP, chaque couche ajoute son en-tête autour des données, comme des enveloppes emboîtées. HTTP → + en-tête TCP (ports) → + en-tête IP (adresses IP) → + en-tête Ethernet (adresses MAC). Le destinataire ouvre les enveloppes dans l'ordre inverse. Wireshark te montre exactement ces enveloppes.

Les trois adresses à ne jamais confondre

- **MAC** (52:54:00:ab:cd:ef) : identifie une carte réseau, sert uniquement sur le réseau local. Ne traverse jamais un routeur.
- **IP** (192.168.1.20) : identifie une machine dans un réseau et permet de la joindre de partout. Reste la même de bout en bout (sauf NAT).
- **Port** (443) : identifie le programme sur la machine. IP + port = **socket** (192.168.1.20:443).

Analogie : l'IP est l'adresse de l'immeuble, le port le numéro d'appartement, la MAC le nom du facteur qui fait le dernier tronçon, et change à chaque tronçon.

Switch vs routeur

- Un **switch** relie les machines d'un même réseau local (LAN). Il apprend quelle MAC est derrière quel port et y envoie les trames.
- Un **routeur** relie des réseaux différents entre eux. Il lit l'IP de destination et choisit la prochaine étape grâce à sa **table de routage**. Ta box Internet est à la fois routeur, switch, point d'accès Wi-Fi, serveur DHCP et NAT.

Topologies et tailles : LAN (maison, bureau), WAN (relie des sites distants), Internet (le réseau des réseaux). Un **VLAN** découpe un même switch en plusieurs réseaux logiques isolés.

4. TCP/IP en détail

TCP/IP est la suite de protocoles d'Internet : **IP** trouve le chemin jusqu'à la bonne machine, **TCP** ou **UDP** livre les données au bon programme. Tout le reste (DNS, DHCP, HTTP, SSH) s'appuie dessus.

4.1 L'adresse IPv4

Une adresse IPv4 fait 32 bits, écrits en 4 octets décimaux : 192.168.1.20. Elle a deux parties : la partie **réseau** (le quartier) et la partie **hôte** (la maison). Le **masque** dit où est la coupure.

- 192.168.1.20/24 = masque 255.255.255.0 : les 24 premiers bits (192.168.1) sont le réseau, le dernier octet est l'hôte.
- Dans chaque réseau, la première adresse est l'**adresse réseau** (192.168.1.0) et la dernière l'adresse de **broadcast** (192.168.1.255) : elles ne sont pas attribuables.
- Nombre d'hôtes = $2^{(32 - \text{préfixe})} - 2$.

Notation CIDR	Masque	Adresses	Hôtes utilisables	Usage typique
/8	255.0.0.0	16 777 216	16 777 214	Grand réseau privé 10.0.0.0/8
/16	255.255.0.0	65 536	65 534	Un VPC AWS entier (10.0.0.0/16)
/24	255.255.255.0	256	254	Un LAN, un sous-réseau AWS
/26	255.255.255.192	64	62	Petit sous-réseau
/28	255.255.255.240	16	14	Le plus petit sous-réseau autorisé par AWS
/30	255.255.255.252	4	2	Lien point à point entre 2 routeurs
/32	255.255.255.255	1	1 (une seule machine)	Règle de pare-feu pour une IP précise

Adresses privées (RFC 1918, non routables sur Internet) : 10.0.0.0/8 , 172.16.0.0/12 , 192.168.0.0/16 . **Spéciales** : 127.0.0.1 (localhost, la machine elle-même), 0.0.0.0 (toutes les interfaces, ou « route par défaut » en 0.0.0.0/0), 169.254.x.x (pas de DHCP trouvé).

Découper en sous-réseaux (subnetting) : prendre 10.0.0.0/16 et le couper en /24 donne 256 sous-réseaux (10.0.0.0/24 , 10.0.1.0/24 ...). C'est exactement ce que tu feras sur AWS : un sous-réseau public et un privé par zone. Entraîne-toi jusqu'à le faire de tête.

4.2 IPv6

IPv4 n'offre que ~4,3 milliards d'adresses, épuisées. **IPv6** utilise 128 bits, écrits en hexadécimal : 2001:db8::1 . Plus besoin de NAT, configuration automatique (SLAAC). Les deux cohabitent aujourd'hui (« double pile »).

4.3 TCP vs UDP

	TCP	UDP
Connexion	Oui, établie avant d'envoyer (handshake)	Non, on envoie directement

	TCP	UDP
Fiabilité	Accusés de réception, retransmission, ordre garanti	Aucune garantie
Contrôle de flux et de congestion	Oui	Non
Vitesse	Plus lent, plus d'en-têtes	Très rapide, léger
Exemples	HTTP/1.1 et 2, SSH, PostgreSQL, SMTP	DNS, DHCP, VoIP, jeux, streaming, HTTP/3 (QUIC)

Le three-way handshake TCP (ouverture de connexion) :

1. Le client envoie **SYN** (« je veux parler »).
2. Le serveur répond **SYN-ACK** (« d'accord, moi aussi »).
3. Le client envoie **ACK** (« c'est parti »). La connexion est établie.

La fermeture se fait avec **FIN / ACK** de chaque côté ; une coupure brutale envoie **RST**. Un port fermé répond RST ; un pare-feu qui bloque ne répond rien (timeout). Cette différence t'aidera à diagnostiquer.

4.4 Les protocoles qui font marcher le réseau

- **ARP** : « qui a l'IP 192.168.1.1 ? donne-moi ta MAC ». Fait le lien IP → MAC sur le réseau local. Commande : `ip neigh`.
- **DHCP** : donne automatiquement à une machine qui arrive son IP, son masque, sa passerelle et son DNS. 4 étapes : Discover, Offer, Request, Ack (« DORA »).
- **DNS** : traduit un nom (`navmarket.lan` , `google.com`) en adresse IP. Le résolveur interroge les serveurs racine, puis ceux du `.com` , puis le serveur faisant autorité pour le domaine. Enregistrements clés : **A** (IPv4), **AAAA** (IPv6), **CNAME** (alias), **MX** (mail), **TXT** , **NS** . Commandes : `dig` , `nslookup` , fichier `/etc/hosts` .
- **Passerelle par défaut** : le routeur à qui la machine envoie tout ce qui n'est pas sur son réseau local. Commande : `ip route` .
- **Routage** : chaque routeur regarde l'IP de destination, cherche la route la plus précise dans sa table et transmet au routeur suivant (saut ou « hop »). Commande : `traceroute` .
- **NAT** : ta box remplace l'IP privée de ta machine par son IP publique en sortie, et retient la correspondance pour renvoyer la réponse. Le **port forwarding** (ou DNAT) fait l'inverse : il ouvre un service interne à Internet.
- **ICMP** : messages de contrôle et d'erreur. `ping` l'utilise.

- **TLS** : chiffre une connexion TCP (HTTPS = HTTP + TLS). Le serveur prouve son identité avec un **certificat** (Let's Encrypt en S4.6).

4.5 Ports à connaître par cœur

Port	Protocole	Service
22	TCP	SSH (et SFTP)
25 / 587	TCP	SMTP (envoi de mail)
53	UDP et TCP	DNS
67 / 68	UDP	DHCP serveur / client
80	TCP	HTTP
123	UDP	NTP (heure)
443	TCP (et UDP pour HTTP/3)	HTTPS
3000	TCP	Next.js / Node en développement (convention)
3306	TCP	MySQL
5432	TCP	PostgreSQL
6379	TCP	Redis
6443	TCP	API Kubernetes
8080	TCP	HTTP alternatif, proxys, applis Java

Ports 0-1023 = ports « bien connus », ouvrables seulement par root. Le client, lui, utilise un **port éphémère** aléatoire (32768-60999 sous Linux).

4.6 Ce qui se passe quand tu tapes `https://navmarket.lan`

1. Le navigateur demande l'IP de `navmarket.lan` au **DNS** (cache, `/etc/hosts`, puis résolveur en UDP 53).
2. Si le serveur est sur le même réseau, **ARP** trouve sa MAC ; sinon ARP trouve la MAC de la **passerelle**.
3. **TCP** ouvre une connexion vers l'IP sur le port 443 (SYN, SYN-ACK, ACK).
4. **TLS** négocie le chiffrement et vérifie le certificat.
5. Le navigateur envoie la requête **HTTP GET /** ; Nginx répond `200 OK` avec le HTML.
6. Chaque paquet traverse les couches et les routeurs, encapsulé et décapsulé à chaque étape.

C'est une question d'entretien classique : sache la raconter en 2 minutes.

4.7 Commandes de diagnostic Linux

`ip a` (interfaces et IP) · `ip route` (routes) · `ping` (joignable ?) · `traceroute` / `mtr` (chemin) · `dig` (DNS) · `ss -tulpn` (ports en écoute) · `curl -v` (requête HTTP détaillée) · `nc -zv hôte port` (port ouvert ?) · `tcpdump` / Wireshark (voir les paquets).

5. La virtualisation

Virtualiser, c'est faire croire à plusieurs systèmes d'exploitation qu'ils ont chacun leur propre machine, alors qu'ils partagent un seul serveur physique. Le logiciel qui réalise ce tour de magie s'appelle l'**hyperviseur**.

5.1 Pourquoi virtualiser

Avant (années 1990-2000), la règle était « une application = un serveur physique », pour éviter les conflits. Résultat : des serveurs utilisés à 10-15 % de leur capacité, des salles pleines, des semaines pour en commander un nouveau. La virtualisation règle cela :

- **Consolidation** : 10 à 20 VM sur un serveur, donc moins de matériel, d'électricité et de place.
- **Isolation** : une VM qui plante ou est piratée n'affecte pas les autres.
- **Rapidité** : créer une VM prend quelques minutes, pas des semaines.
- **Mobilité** : une VM est un ensemble de fichiers ; on peut la copier, la sauvegarder, la déplacer à chaud vers un autre serveur (**live migration**).
- **Snapshots** : photographier l'état d'une VM avant une manipulation risquée et y revenir. Parfait pour tes labs.

5.2 Les deux types d'hyperviseurs

	Type 1 « bare metal »	Type 2 « hébergé »
Où il tourne	Directement sur le matériel, c'est lui l'OS	Comme une application, au-dessus d'un OS classique
Performance	Maximale	Plus faible (une couche en plus)
Usage	Datacenters, cloud, production	Poste de travail, tests, apprentissage
Exemples	VMware ESXi, Microsoft Hyper-V, Xen, KVM (intégré au noyau Linux), Proxmox VE (basé sur KVM)	VirtualBox, VMware Workstation, Parallels

Cas de KVM : c'est un module du noyau Linux qui transforme Linux lui-même en hyperviseur de type 1. **QEMU** émule les périphériques, **libvirt** fournit une API commune et **virt-manager** l'interface graphique que tu utilises. AWS EC2 tourne aujourd'hui sur **Nitro**, un hyperviseur dérivé de KVM.

5.3 Comment ça marche

- **CPU** : grâce à Intel VT-x / AMD-V, le processeur sait exécuter directement les instructions de la VM et rend la main à l'hyperviseur pour les opérations sensibles. Chaque VM voit des **vCPU**.
- **Mémoire** : l'hyperviseur attribue à chaque VM une plage de RAM qu'elle croit physique (extensions EPT / NPT pour la traduction rapide). On peut faire du **surengagement** (donner plus de RAM au total qu'il n'en existe), avec prudence.
- **Disque** : le disque d'une VM est un fichier (`.qcow2` pour KVM, `.vmdk` pour VMware, `.vhdx` pour Hyper-V), en allocation fine (grossit au besoin) ou fixe.
- **Périphériques** : émulés (lents) ou **paravirtualisés** (pilotes `virtio`, rapides, car l'OS invité sait qu'il est virtualisé).
- **Vocabulaire** : la machine physique = l'**hôte** (host) ; la VM = l'**invité** (guest).

5.4 Le réseau virtuel

L'hyperviseur crée des **switchs virtuels** (bridges Linux, Open vSwitch). Trois modes à connaître :

- **NAT** : la VM sort sur Internet en empruntant l'IP de l'hôte, mais n'est pas joignable de l'extérieur. Mode par défaut de virt-manager (réseau `default`, `192.168.122.0/24`).
- **Pont (bridge)** : la VM est branchée sur ton vrai réseau et y obtient sa propre IP, comme un PC de plus.

- **Réseau isolé / hôte uniquement** : les VM se parlent entre elles et avec l'hôte, sans Internet. Idéal pour simuler un réseau d'entreprise.

5.5 Au-delà des VM

On virtualise aussi le **stockage** (SAN, Ceph : des disques de plusieurs machines vus comme un seul) et le **réseau** (SDN : le réseau piloté par logiciel, VLAN, VXLAN). Quand calcul, stockage et réseau sont tous virtualisés et pilotés par logiciel, on obtient un **datacenter défini par logiciel** : la fondation directe du cloud.

Acteurs du marché : VMware vSphere (leader historique, racheté par Broadcom en 2023, ce qui a poussé beaucoup d'entreprises vers Proxmox, Hyper-V ou Nutanix), Microsoft Hyper-V, Proxmox VE, Nutanix AHV, KVM/OpenStack, Citrix XenServer.

6. De la virtualisation au cloud

Le cloud n'est pas une technologie nouvelle : c'est la virtualisation **plus** l'automatisation, le libre-service et la facturation à l'usage, à l'échelle de datacenters géants. La virtualisation a rendu les serveurs logiciels ; le cloud a rendu ces serveurs acheteables par un simple appel d'API.

6.1 Ce que la virtualisation seule ne réglait pas

Avec VMware dans son propre datacenter, une entreprise devait encore :

- acheter et renouveler le matériel tous les 3 à 5 ans, et le dimensionner pour le pic (Black Friday) même s'il dormait le reste de l'année ;
- gérer bâtiment, électricité, climatisation, réseau, sécurité physique ;
- passer par un ticket à l'équipe infrastructure pour obtenir une VM (souvent des jours) ;
- payer d'avance (**CAPEX**, investissement) au lieu de payer ce qu'elle consomme.

6.2 Les trois ingrédients ajoutés par le cloud

1. **Le libre-service par API** : tout se crée en quelques secondes, par console web, ligne de commande ou code (`aws ec2 run-instances`, Terraform). C'est ce qui rend possible l'**Infrastructure as Code** et le DevOps.
2. **L'élasticité** : ajouter 50 serveurs pour 2 heures puis les rendre. L'**auto-scaling** le fait automatiquement selon la charge.
3. **Le paiement à l'usage (OPEX, dépense de fonctionnement)** : à la seconde ou à l'heure, sans engagement.

S'ajoutent l'échelle (des millions de serveurs mutualisés entre clients, donc moins chers), la portée mondiale (des régions sur tous les continents) et des **services managés** : le fournisseur gère pour toi la base de données, les sauvegardes, les mises à jour.

6.3 Les grandes dates

Année	Événement	Pourquoi c'est important
1972	IBM VM/370	Première virtualisation commerciale, sur mainframe
1999-2001	VMware Workstation puis ESX	La virtualisation arrive sur les serveurs x86 ordinaires
2003	Xen (libre)	Hyperviseur qui fera tourner AWS EC2 à ses débuts
2005-2006	Intel VT-x et AMD-V	Le processeur aide l'hyperviseur : virtualisation rapide et simple
2006	AWS lance S3 puis EC2	Naissance du cloud public moderne : des VM louables à l'heure par API
2007	KVM intégré au noyau Linux	Linux devient hyperviseur, gratuitement
2008-2010	Google App Engine, Microsoft Azure, OpenStack	Concurrence et cloud privé libre
2011	Définition du cloud par le NIST (SP 800-145)	Vocabulaire officiel : IaaS, PaaS, SaaS
2013	Docker	Conteneurs simples à utiliser : on livre l'application, pas la VM
2014	Kubernetes (Google), AWS Lambda	Orchestration de conteneurs et serverless
2017	AWS Nitro	Hyperviseur maison dérivé de KVM, matériel dédié
Années 2020	Cloud natif, multi-cloud, IA	Kubernetes devient le standard, le cloud porte l'essentiel des nouveaux projets

6.4 Le fil logique à retenir

Serveur physique dédié → **VM** (on partage la machine) → **cloud IaaS** (on loue des VM à la demande) → **PaaS et services managés** (on ne gère plus l'OS) → **conteneurs** (on livre juste l'application et ses dépendances) → **Kubernetes** (on orchestre des milliers de conteneurs) → **serverless** (on livre juste une fonction). À chaque étape, on délègue une couche de plus et on gagne en vitesse, au prix de moins de contrôle.

7. Le cloud

Le cloud, c'est louer à la demande, via Internet et par API, de la puissance de calcul, du stockage, du réseau et des services tout faits, en payant ce qu'on consomme. Le « nuage » est en réalité très physique : des centaines de datacenters remplis de serveurs.

7.1 La définition officielle (NIST) : 5 caractéristiques

1. **Libre-service à la demande** : tu crées tes ressources toi-même, sans parler à personne.
2. **Accès réseau large** : accessible depuis n'importe où, par Internet.
3. **Mutualisation des ressources** : le matériel est partagé entre de nombreux clients (multi-tenant), isolés les uns des autres.
4. **Élasticité rapide** : on monte ou descend en capacité en quelques minutes.
5. **Service mesuré** : tout est compté et facturé à l'usage.

7.2 Les modèles de service : qui gère quoi

Couche	On-premise (chez soi)	IaaS	PaaS	SaaS
Application et données	Toi	Toi	Toi	Fournisseur (tu gères tes données et tes accès)
Runtime, middleware	Toi	Toi	Fournisseur	Fournisseur
Système d'exploitation	Toi	Toi	Fournisseur	Fournisseur
Virtualisation	Toi	Fournisseur	Fournisseur	Fournisseur

Couche	On-premise (chez soi)	IaaS	PaaS	SaaS
Serveurs, stockage, réseau	Toi	Fournisseur	Fournisseur	Fournisseur
Datacenter, électricité	Toi	Fournisseur	Fournisseur	Fournisseur
Exemples	Ton serveur au bureau	AWS EC2, Azure VM, Oracle Cloud (ton VPS)	AWS Elastic Beanstalk, Heroku, Vercel, Google App Engine	Gmail, Microsoft 365, Salesforce, GitHub

Analogie de la pizza : faite maison (on-premise), kit à cuire (IaaS), livrée (PaaS), au restaurant (SaaS). Entre les deux s'ajoutent le **CaaS** (conteneurs managés : ECS, EKS) et le **FaaS / serverless** (tu livres une fonction : AWS Lambda).

7.3 Les modèles de déploiement

- **Cloud public** : infrastructure partagée d'un fournisseur (AWS, Azure, Google Cloud, OVHcloud, Scaleway).
- **Cloud privé** : réservé à une seule organisation, chez elle ou hébergé (OpenStack, VMware, Proxmox avec automatisation).
- **Hybride** : un mélange relié des deux (données sensibles en interne, le reste dans le cloud).
- **Multi-cloud** : plusieurs fournisseurs publics en même temps.
- **Cloud souverain** : hébergé et opéré sous juridiction européenne (label SecNumCloud de l'ANSSI en France).

7.4 L'infrastructure mondiale (exemple AWS)

- **Région** : une zone géographique (`eu-west-3` = Paris). Tes données y restent sauf si tu les déplaces.
- **Zone de disponibilité (AZ)** : un ou plusieurs datacenters isolés dans une région (`eu-west-3a`, `b`, `c`). Mettre ses serveurs sur 2 AZ minimum = résister à la panne d'un datacenter.
- **Points de présence (edge)** : des serveurs proches des utilisateurs pour le cache (CloudFront, CDN).

7.5 Le modèle de responsabilité partagée

AWS est responsable de la sécurité **du** cloud (bâtiments, matériel, hyperviseur, réseau mondial). Toi, tu es responsable de la sécurité **dans** le cloud : tes données, les accès IAM, les règles de pare-feu, les mises à jour de l'OS de tes VM, le chiffrement. La plupart des fuites viennent d'erreurs du client (un bucket S3 public, une clé d'accès poussée sur GitHub).

7.6 Les services de base et leurs équivalents

Besoin	AWS	Azure	Google Cloud	Ce que ça remplace
Machines virtuelles	EC2	Virtual Machines	Compute Engine	Tes VM KVM
Réseau privé	VPC (sous-réseaux, tables de routage, passerelle Internet, NAT Gateway)	Virtual Network	VPC	Ton LAN, ta box
Pare-feu	Security Groups, Network ACL	Network Security Groups	Firewall rules	<code>ufw</code> , <code>iptables</code>
Stockage objet	S3	Blob Storage	Cloud Storage	Un partage de fichiers illimité
Disques de VM	EBS	Managed Disks	Persistent Disk	Le fichier <code>.qcow2</code>
Base de données managée	RDS, Aurora	Azure SQL, Database for PostgreSQL	Cloud SQL	Ton PostgreSQL installé à la main
Identités et droits	IAM	Entra ID	Cloud IAM	Utilisateurs Linux et <code>sudo</code>
DNS	Route 53	Azure DNS	Cloud DNS	Ton serveur DNS <code>navmarket.lan</code>
Équilibrage de charge	ELB (ALB, NLB)	Load Balancer, Application Gateway	Cloud Load Balancing	Nginx en reverse proxy
Conteneurs	ECS, Fargate, EKS	Container Apps, AKS	Cloud Run, GKE	Docker et Kubernetes chez toi

Besoin	AWS	Azure	Google Cloud	Ce que ça remplace
Serverless	Lambda	Functions	Cloud Run functions	Un script lancé sur un serveur
Supervision	CloudWatch	Azure Monitor	Cloud Monitoring	Prometheus, Grafana, journalctl
Infrastructure as Code	CloudFormation (ou Terraform)	Bicep (ou Terraform)	Terraform	Tes clics dans virt-manager

7.7 Avantages et pièges

- **Pour** : vitesse, pas de matériel à acheter, élasticité, portée mondiale, services managés, haute disponibilité accessible à tous.
- **Contre** : la facture peut exploser si on oublie des ressources (active les **alertes de budget** dès le premier jour), dépendance au fournisseur (**vendor lock-in**), coûts de sortie des données, souveraineté des données, complexité des droits IAM. Le métier qui pilote les coûts s'appelle **FinOps**.

8. Conteneurs et Docker

Un conteneur est un processus Linux ordinaire, enfermé dans une bulle qui lui fait croire qu'il est seul sur la machine. Contrairement à une VM, il **n'a pas son propre noyau** : tous les conteneurs partagent le noyau de l'hôte. D'où un démarrage en une seconde et quelques Mo au lieu de plusieurs Go.

8.1 Le problème résolu

« Chez moi ça marche » : l'application fonctionne sur le poste du développeur mais pas en production, à cause d'une version de Node, d'une bibliothèque ou d'une config différente. Le conteneur emballe l'application **avec toutes ses dépendances** dans une image unique qui tourne à l'identique partout : ton PC, le VPS, AWS.

8.2 Comment Linux isole un conteneur

- **Namespaces** : ce que le processus peut **voir**. Il a ses propres PID (il se croit PID 1), son réseau, ses points de montage, son nom d'hôte, ses utilisateurs.
- **cgroups** : ce que le processus peut **consommer**. Limites de CPU, RAM, disque (`docker run --memory 512m`).

- **Système de fichiers en couches (OverlayFS)** : l'image est faite de couches en lecture seule, le conteneur ajoute une couche fine en écriture.

Conséquence importante : un conteneur Linux a besoin d'un noyau Linux. Sur Windows ou macOS, Docker Desktop lance en fait une petite VM Linux (WSL2 sous Windows). Et l'isolation est plus faible qu'une VM, puisque le noyau est partagé.

8.3 Le vocabulaire Docker

Terme	Définition	Analogie
Dockerfile	Recette texte qui décrit comment construire l'image	La recette
Image	Paquet immuable : un système de fichiers minimal + ton app + ses dépendances + la commande de démarrage	Le plat surgelé (ou une classe en programmation)
Conteneur	Une image en cours d'exécution ; éphémère, jetable	Le plat réchauffé (ou un objet, instance de la classe)
Registre	Entrepôt d'images : Docker Hub, GHCR (GitHub, ton S11), Amazon ECR	Le supermarché
Volume	Stockage qui survit à la suppression du conteneur	Le disque dur externe
Réseau Docker	Switch virtuel entre conteneurs, avec DNS par nom de service	Le LAN des conteneurs
Docker Compose	Fichier YAML qui décrit plusieurs conteneurs ensemble (front, API, base)	Le menu complet

8.4 Un exemple concret

```
FROM node:22-alpine          # image de base légère
WORKDIR /app
COPY package*.json ./
RUN npm ci                   # chaque instruction = une couche (mise en
                             # cache)
COPY . .
RUN npm run build
EXPOSE 3000
CMD ["node", "dist/main.js"] # processus principal du conteneur
```

Cycle de vie : `docker build -t navmarket-api .` → `docker run -d -p 8080:3000 navmarket-api` (le port 8080 de l'hôte renvoie vers le 3000 du conteneur) → `docker push ghcr.io/...` → sur le serveur, `docker pull` puis `docker run`. Autres commandes : `docker ps`, `docker logs`, `docker exec -it <id> sh`, `docker compose up -d`.

8.5 Les principes à respecter

- **Un processus principal par conteneur** (l'API d'un côté, PostgreSQL de l'autre).
- **Immuable et jetable** : on ne corrige pas un conteneur en place, on reconstruit l'image et on remplace le conteneur.
- **Les données dans des volumes**, jamais dans le conteneur.
- **La config par variables d'environnement**, les secrets jamais dans l'image.
- **Images petites et sûres** : base `alpine` ou `slim`, build multi-étapes, utilisateur non root.

Docker n'est qu'un outil parmi d'autres : le format d'image est standardisé par l'**OCI**, et d'autres moteurs l'exécutent (containerd, CRI-O, Podman). Kubernetes utilise containerd ou CRI-O, pas Docker lui-même, mais tes images Docker y tournent sans changement.

9. Kubernetes

Kubernetes (K8s) est un chef d'orchestre : tu lui décris l'état voulu (« 3 copies de mon API, accessibles sur le port 80 ») et il fait tout pour que la réalité y corresponde, en continu, sur un ensemble de machines. Docker lance des conteneurs sur une machine ; Kubernetes les pilote sur un cluster.

9.1 Les problèmes qu'il règle

Avec 3 conteneurs sur un serveur, Docker Compose suffit. Avec 300 conteneurs sur 20 serveurs, il faut répondre à : sur quelle machine placer chaque conteneur ? Que faire si un conteneur ou un serveur meurt ? Comment ajouter des copies quand la charge monte ?

Comment mettre à jour sans coupure ? Comment les conteneurs se trouvent-ils entre eux ? Kubernetes automatise toutes ces réponses.

9.2 L'architecture d'un cluster

- **Plan de contrôle (control plane), le cerveau :**
 - **kube-apiserver** : la porte d'entrée, tout passe par son API (port 6443) ; `kubectl` lui parle.
 - **etcd** : la base clé-valeur qui stocke l'état du cluster.
 - **kube-scheduler** : choisit sur quel nœud placer chaque pod.
 - **controller-manager** : les boucles qui comparent l'état voulu et l'état réel et corrigent l'écart.
- **Nœuds de travail (workers), les muscles (des VM ou des machines physiques) :**
 - **kubelet** : l'agent qui lance et surveille les pods sur le nœud.
 - **runtime de conteneurs (containerd)** : exécute réellement les conteneurs.
 - **kube-proxy** : gère les règles réseau des Services.

9.3 Les objets essentiels

Objet	Rôle
Pod	Plus petite unité : un ou plusieurs conteneurs qui partagent la même IP et des volumes. Mortel et remplaçable.
Deployment	Décrit combien de copies (replicas) d'un pod faire tourner et comment les mettre à jour (rolling update, rollback).
ReplicaSet	Géré par le Deployment, il maintient le bon nombre de pods.
Service	Adresse stable (IP + nom DNS) devant un groupe de pods changeants, avec répartition de charge. Types : ClusterIP (interne), NodePort, LoadBalancer.
Ingress / Gateway API	Point d'entrée HTTP(S) depuis l'extérieur, routage par nom de domaine et chemin, TLS.
ConfigMap / Secret	Configuration et secrets injectés dans les pods.
PersistentVolume / PVC	Stockage durable pour les applis avec état.

Objet	Rôle
StatefulSet	Pour les bases de données : identité et stockage stables par pod.
DaemonSet	Un pod sur chaque nœud (agents de logs, de supervision).
Namespace	Découpe logique du cluster (dev, prod, équipes).
HorizontalPodAutoscaler	Ajoute ou retire des pods selon le CPU ou d'autres métriques.

9.4 Le principe déclaratif

On n'écrit pas « lance ceci », on décrit « je veux ceci » dans un fichier YAML, puis `kubectl apply -f deployment.yaml`. Si un pod meurt, le controller voit 2 copies au lieu de 3 et en recrée une : c'est l'**auto-réparation**. Ce fichier se versionne dans Git, ce qui mène au **GitOps** (Argo CD, Flux).

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: navmarket-api
spec:
  replicas: 3                # état voulu : 3 pods
  selector:
    matchLabels: { app: navmarket-api }
  template:
    metadata:
      labels: { app: navmarket-api }
    spec:
      containers:
        - name: api
          image: ghcr.io/navatech/navmarket-api:1.0.0
          ports: [{ containerPort: 3000 }]
```

9.5 Où faire tourner Kubernetes

- **En local pour apprendre** : kind, minikube, k3s (léger, parfait pour un homelab).
- **Installé soi-même** : kubeadm (c'est ce qu'on pratique pour la certification CKA).
- **Managé dans le cloud** : Amazon EKS, Azure AKS, Google GKE. Le fournisseur gère le plan de contrôle, tu gères les nœuds et tes applications.

Quand ne pas utiliser Kubernetes : petite application, petite équipe, peu de trafic. Docker Compose sur un VPS ou un service comme ECS Fargate est souvent plus simple et moins cher. Kubernetes apporte de la puissance, mais aussi beaucoup de complexité.

10. Virtualisation vs Docker vs Kubernetes vs cloud

Ces quatre notions ne sont pas concurrentes : elles répondent à des questions différentes et s'empilent. La virtualisation découpe une machine, Docker emballe une application, Kubernetes pilote des conteneurs sur plusieurs machines, et le cloud te loue tout cela à la demande.

Une VM embarque son propre OS ; un conteneur partage le noyau de l'hôte



serveur physique, VM et conteneurs · ce que chacun empile

La différence clé tient à une ligne : chaque VM transporte un OS complet, alors que les conteneurs partagent le noyau de l'hôte et n'emportent que l'application et ses bibliothèques.

Critère	Machine virtuelle	Conteneur (Docker)	Kubernetes	Cloud
Question à laquelle il répond	Comment faire tourner plusieurs OS sur un serveur ?	Comment livrer une appli qui tourne pareil partout ?	Comment piloter des centaines de conteneurs sur plusieurs machines ?	Comment obtenir tout cela sans acheter ni gérer de matériel ?

Critère	Machine virtuelle	Conteneur (Docker)	Kubernetes	Cloud
Ce qu'on manipule	Une machine complète	Une application et ses dépendances	Un état voulu (YAML) pour tout un cluster	Des services loués par API
Niveau de virtualisation	Le matériel	Le système d'exploitation	Un cluster de machines	Un datacenter entier
Taille et démarrage	Go, minutes	Mo, secondes	Dépend des conteneurs	Secondes à minutes
Isolation	Forte (noyau dédié)	Plus légère (noyau partagé)	Celle des conteneurs, plus des politiques réseau	Celle choisie par le fournisseur, plus la tienne (IAM, réseaux)
Outils	KVM, VMware, Hyper-V, Proxmox, VirtualBox	Docker, Podman, containerd	Kubernetes, k3s, EKS, AKS, GKE	AWS, Azure, Google Cloud, OVHcloud
Bon pour	Faire tourner des OS différents, isoler fortement, applis anciennes	Microservices, CI/CD, environnement de dev identique à la prod	Grosses applis en production, haute dispo, montée en charge automatique	Démarrer vite, absorber des pics, services managés, portée mondiale
Dans ton parcours	S7 (KVM, Proxmox), S8 (Vagrant)	S11	S19-21 (CKA)	S13-16 (AWS), S17-18 (Terraform)

Comment tout s'empile en vrai : sur Amazon EKS, ton conteneur NavMarket tourne dans un **pod** Kubernetes, sur un **nœud** qui est une **VM** EC2, créée par l'hyperviseur **Nitro**, sur un **serveur physique** dans un datacenter AWS de la région Paris. Les quatre couches sont présentes en même temps.

Quand choisir quoi, en résumé

- Un OS entier à tester ou isoler (Windows, un pare-feu, un lab qui casse) → **VM**.
- Une appli à livrer de façon reproductible → **conteneur** ; plusieurs conteneurs sur un serveur → **Docker Compose**.

- Beaucoup de conteneurs, plusieurs serveurs, zéro coupure → **Kubernetes** (ou ECS si tu veux plus simple sur AWS).
- Pas envie d'acheter ni de gérer le matériel, besoin d'élasticité → **cloud**, quelle que soit la technologie du dessus.

11. Sources fiables pour approfondir

Si tu ne gardes qu'un seul site en français : la [formation de Stéphane Robert](#), écrite pour les DevOps, couvre réseau, Linux, conteneurs, Kubernetes et AWS dans l'ordre de ton parcours. Pour le reste, privilégie toujours la documentation officielle de l'outil.

Thème	Ressource	Langue	Format
Bases de l'informatique (matériel, OS)	Crash Course Computer Science, épisode 1 (série de 40 vidéos de ~12 min)	Anglais, sous-titres FR	Vidéo
Linux	Linux Journey	Anglais	Cours interactif gratuit
Réseau, débutant	Les réseaux de zéro (Zeste de Savoir)	Français	Tutoriel complet
Réseau pour DevOps	Fondamentaux des réseaux (Stéphane Robert) et Modèle OSI et TCP/IP	Français	Guide
Diagnostic réseau Linux	Diagnostiquer un problème réseau sous Linux	Français	Guide pratique
Réseau, explications courtes	Cloudflare Learning : le modèle OSI et Qu'est-ce qu'un protocole ?	Français ou anglais	Articles
Réseau, cours structuré	Cisco Networking Academy : Networking Basics	Anglais	Cours gratuit avec badge
Virtualisation	Red Hat : un hyperviseur, qu'est-ce que c'est ? et KVM, qu'est-ce que c'est ?	Français (choisir FR sur le site)	Articles
Cloud, définition officielle	NIST SP 800-145 (PDF, 7 pages)	Anglais	Norme

Thème	Ressource	Langue	Format
Cloud AWS	Qu'est-ce que le cloud computing ? (AWS) et Fondamentaux du cloud AWS	Français	Livre blanc, parcours
Responsabilité partagée	Modèle de responsabilité partagée AWS	Français	Livre blanc
VM vs conteneurs	Google Cloud : conteneurs et VM , AWS : différence entre conteneurs et VM , Red Hat : conteneurs et machines virtuelles	Français	Articles
Docker	What is Docker? (doc officielle) puis Docker workshop	Anglais	Doc + atelier pratique
Kubernetes	Kubernetes : Overview puis Learn Kubernetes Basics	Anglais (FR partiel sur le site)	Doc + tutoriel interactif
Kubernetes en français	Formation Kubernetes (Stéphane Robert)	Français	Guide complet

Ordre de lecture conseillé pour démarrer cette semaine : Crash Course épisodes 1 à ~20 en fond (matériel et OS), Linux Journey en parallèle de S1-2, puis Les réseaux de zéro avant S3. Les ressources cloud, Docker et Kubernetes attendent leur semaine.

12. Auto-évaluation

Tu maîtrises les bases quand tu sais répondre à voix haute, sans notes, à ces questions. Coche-les au fil des semaines.

Matériel et OS

- ☐ Quel rôle jouent le CPU, la RAM et le disque ? Que se passe-t-il quand la RAM est pleine ?
- ☐ Décris le démarrage d'une machine Linux, de l'UEFI jusqu'à systemd.
- ☐ Qu'est-ce qu'un appel système, et pourquoi un programme ne parle-t-il pas directement au matériel ?
- ☐ Quelle est la différence entre un programme, un processus et un service ?

Réseau

- ☐ Cite les couches TCP/IP et un protocole par couche.
- ☐ MAC, IP, port : à quoi sert chacun, et lequel change à chaque routeur ?
- ☐ Combien d'hôtes dans un /26 ? Découpe 10.0.0.0/16 en 4 sous-réseaux égaux.
- ☐ TCP ou UDP pour DNS, SSH, la visio ? Pourquoi ?
- ☐ Explique le three-way handshake et la différence entre une réponse RST et un timeout.
- ☐ Raconte ce qui se passe quand tu tapes une URL dans le navigateur.
- ☐ À quoi servent ARP, DHCP, DNS et NAT ?

Virtualisation et cloud

- ☐ Différence entre hyperviseur de type 1 et de type 2 ? Dans quelle catégorie est KVM ?
- ☐ Différence entre les modes réseau NAT et pont d'une VM ?
- ☐ Quels sont les 3 ingrédients qui font passer de la virtualisation au cloud ?
- ☐ IaaS, PaaS, SaaS : qui gère l'OS dans chaque cas ? Donne un exemple de chaque.
- ☐ Région vs zone de disponibilité : pourquoi déployer sur 2 AZ ?
- ☐ Dans le modèle de responsabilité partagée, qui patche l'OS d'une instance EC2 ?

Conteneurs et Kubernetes

- ☐ Pourquoi un conteneur démarre-t-il en une seconde et une VM en une minute ?
- ☐ Image vs conteneur ? Pourquoi les données doivent-elles aller dans un volume ?
- ☐ Que sont les namespaces et les cgroups ?
- ☐ Pod, Deployment, Service : explique le rôle de chacun.
- ☐ Que fait Kubernetes si un nœud tombe en panne ?
- ☐ Pour la boutique NavMarket, quand choisirais-tu Docker Compose sur un VPS, ECS Fargate ou EKS ?